

LOTERIA BINARIA

Manual intercalado de desarrollo y prompts

En cada fase: preparar, preguntar a la IA, editar, comprobar, capturar y continuar

Documento	MI-T3-LB-DJANGO
Version	4.0.0
Proyecto	Loteria Binaria - Taller #3 Django
Fecha	30 de julio de 2026

Taller #3 - Desarrollo de un Sistema Web con Django.

Proyecto: Loteria Binaria - primera fase SQLite, segunda fase MySQL.

Los prompts ya no estan separados: aparecen exactamente en el punto donde deben utilizarse.

Perfil obligatorio del taller

La entrega se desarrolla primero con SQLite y despues se migra a MySQL. La interfaz usa Bootstrap 5.3; no se acepta una entrega basada unicamente en CSS personalizado. La IA se usa por etapas y el estudiante ejecuta, revisa, explica y documenta cada cambio.

Que cambia en esta version

Estructura realmente entrelazada

Ya no existe una guía arriba y un banco de prompts abajo. Cada fase contiene las acciones manuales, el prompt exacto, los archivos que debes editar, los comandos de comprobación, la evidencia y la indicación precisa de cómo proseguir.

- Cada prompt recuerda el contexto del proyecto y restringe a la IA para que no mezcle otros proyectos ni invente funciones.
- Los prompts de interfaz enumeran componentes, páginas, responsive, Bootstrap y elementos obligatorios.
- Los prompts técnicos recuerdan las reglas de roles, montos, lotería, historial, SQLite y MySQL.
- Las tareas del estudiante aparecen antes y después de cada prompt, como en la Guía Taller 3 VideoClub.
- Cada bloque termina con NO AVANZAR SI y el siguiente prompt autorizado.

Fuentes y orden de autoridad

Prioridad	Fuente	Función dentro de este manual
1	Enunciado del Taller #3	Define SQLite primero, MySQL después, tres CRUD, framework visual, mejora adicional, uso de IA y entrega.
2	Reglas Maestras v1.1.0	Define comportamiento, roles, montos, lotería, seguridad y exclusiones del MVP.
3	Plan Técnico v1.1.0	Define apps, modelos, servicios, fases, pruebas y despliegue.
4	Matriz de Trazabilidad v1.1.0	Indica que debe comprobarse y que evidencia demuestra cada regla.
5	Diseño de Interfaz v1.0.0	Define páginas, navegación, componentes, responsive y comportamiento por modo.
6	Auditoría de Coherencia v1.0.0	Resuelve contradicciones del ZIP y clasifica reglas antiguas.
7	Guía Taller 3 VideoClub	Aporta el procedimiento pedagógico: preguntar, editar, migrar, admin, forms, views, urls y templates.
8	ZIP del frontend antiguo	Solo referencia visual y de flujos; nunca fuente de verdad de negocio.

Rutina obligatoria al abrir una terminal nueva

.env no es un comando ni un archivo que se edita

Es una carpeta de entorno virtual. Se crea una sola vez. Debes activarla cada vez que abras una terminal nueva. Mientras trabajas en la misma terminal, no la desactives entre prompts.

```
cd <RUTA_DE_TU_PROYECTO_TALLER_3>
.venv\Scripts\Activate.ps1
python --version
python -m django --version
```

- El prompt de PowerShell debe comenzar con (.venv).
- Debes estar en la carpeta que contiene manage.py para ejecutar comandos Django.
- manage.py normalmente no se edita: se usa para check, makemigrations, migrate, createsuperuser, runserver y test.
- Si todavía no existe manage.py, solo ejecuta las verificaciones de Python/Django y sigue la fase de creación del proyecto.

Rutina después de cada cambio de código

```
python manage.py check
# si cambiaste models.py:
python manage.py makemigrations <app>
python manage.py sqlmigrate <app> <numero_migracion>
python manage.py migrate
```

```
# si hay pruebas de la fase:
python manage.py test <app_o_ruta_de_tests>
```

Orden correcto

Primero editar, luego check, despues revisar makemigrations/sqlmigrate, aplicar migrate y finalmente ejecutar pruebas y abrir la interfaz. No ejecutes migrate a ciegas si la migracion contiene campos o borrados inesperados.

Arquitectura y alcance evaluable

App	Responsabilidad	Tipo de desarrollo en el taller
accounts	User personalizado, terminos, login, grupos y modo activo	CRUD evaluable de usuarios/terminos + autenticacion.
core	Landing, dashboards, navegacion y auditoria	Paginas y consultas; auditoria sin eliminacion.
finance	Wallets, movimientos y operaciones simuladas	Listados y servicios; nunca CRUD generico de saldo.
vendors	Perfil vendedor y solicitudes Cliente-Vendedor	CRUD evaluable de VendorProfile + flujo de consulta.
lottery	Productos, eventos, boletos y resultados	CRUD evaluable de productos/eventos; tickets/resultados protegidos.

Tres CRUD que demuestran la rubrica

Accounts, Vendors y Lottery deben demostrar crear, listar, ver, editar y eliminar de manera segura. Cuando existe historia, la D se cumple mediante eliminacion protegida o desactivacion logica, no borrando registros sensibles.

FASE 0

Analisis, IA y diseno antes de programar

Cumplir los puntos 2.1 a 2.5 del enunciado y cerrar el alcance.

Resultado que debe existir al terminar

Documentos de contexto, apreciacion personal, ER, SQL temporal, lista de apps, modelo consolidado, plan por apps y auditorias.

Secuencia intercalada de esta fase

Paso	Accion del estudiante	Prompt / archivo	Comprobacion para seguir
0.1	Crear carpetas docs/, docs/sql/ y evidencias/00_analisis/.	Accion manual	Carpetas visibles y respaldo del ZIP.
0.2	Fijar autoridad y contradicciones.	P-00	La IA confirma SQLite -> MySQL y no genera codigo.
0.3	Obtener alcance y minimo aceptable.	P-01	Separacion entre minimo, plus y fuera de alcance.
0.4	Redactar apreciacion personal.	P-02	Texto escrito con palabras propias.
0.5	Definir ER y validarlo.	P-03 y P-04	ER aprobado y base temporal eliminada.
0.6	Definir apps, referencia consolidada y plan.	P-05 a P-07	No se ha migrado todavia.
0.7	Auditar ZIP y repositorio.	P-08 y P-09	Mapa de interfaz y estado real del codigo.

0.1 - Preparar la carpeta documental

1. Crea una copia separada del proyecto para el Taller #3. No trabajes sobre el proyecto empresarial.
2. Copia dentro de docs/referencias/ el enunciado, la guia docente y los cinco documentos canonicos.
3. Conserva el ZIP antiguo sin modificar dentro de respaldo_frontend/.
4. Crea evidencias/00_analisis/ para capturas del ER, Workbench temporal y respuestas de IA.

Donde crear o editar

- docs/
- docs/sql/
- docs/referencias/
- evidencias/00_analisis/
- respaldo_frontend/

Evidencia que debes guardar

- [] Arbol de carpetas.
- [] Copia intacta del ZIP.

**NO PROSIGAS
TODAVIA**

No tienes una copia separada del Taller #3 o no puedes identificar cual es el ZIP mas reciente.

P-00

Contexto maestro obligatorio

Fase / uso

Todas - Fijar fuentes, restricciones y formato de respuesta.

ANTES DE ENVIAR EL PROMPT

- [] Adjuntar el enunciado del Taller #3.
- [] Adjuntar los cinco MD canonicos.
- [] Adjuntar el ZIP legado y el repositorio mas reciente.

PROMPT PARA COPIAR

Estoy desarrollando el Taller #3 de "Loteria Binaria" con Django. Usa esta autoridad:

- 1) enunciado del Taller #3 para forma de entrega;
- 2) cinco documentos canonicos del MVP Django para reglas de negocio e interfaz;
- 3) Guia Taller 3 VideoClub para el procedimiento pedagogico;
- 4) ZIP legado solo como referencia visual y de flujos.

Perfil obligatorio del taller: primera fase SQLite, segunda fase MySQL, Bootstrap 5.3 como framework de interfaz, al menos tres apps con CRUD completo y una mejora adicional real. No uses PostgreSQL en esta entrega. No generes todo el proyecto de una vez. Trabaja solo la tarea solicitada, revisa los archivos reales adjuntos y entrega: diagnostico, archivos exactos, codigo completo solo de esos

archivos, comandos, pruebas y puerta de salida.

Antes de proponer cambios, repite el alcance, identifica las contradicciones entre taller, documentos y ZIP, y confirma estas decisiones: SQLite -> MySQL; Bootstrap 5.3; cinco apps canonicas; CRUD evaluable en accounts, vendors y lottery; login/roles como mejora; operaciones sensibles sin delete generico. No escribas codigo todavia.

ARCHIVOS ESPERADOS

- Ningun archivo; solo una declaracion de alcance.

DESPUES DE RECIBIR LA RESPUESTA

1. Comparar la respuesta con el enunciado y las baselines.
2. Corregir cualquier mezcla con el monorepo empresarial o PostgreSQL.
3. Guardar la respuesta en docs/CONTEXTO_IA.md.

EVIDENCIA

[] Respuesta de contexto guardada.

NO AVANZAR SI

La IA propone PostgreSQL, Tailwind y Bootstrap juntos, pagos reales, API obligatoria o proyecto completo.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continue con P-01. No envie dos prompts de implementacion al mismo tiempo.

P-01

Propuesta de alcance y minimo aceptable

Fase / uso

2.1 del enunciado - Obtener una propuesta que el estudiante pueda analizar.

ANTES DE ENVIAR EL PROMPT

- [] P-00 aprobado.
- [] Tener el enunciado visible.
- [] No pedir codigo.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Estoy desarrollando el Taller #3 de "Loteria Binaria" con Django. Usa esta autoridad:

- 1) enunciado del Taller #3 para forma de entrega;
- 2) cinco documentos canonicos del MVP Django para reglas de negocio e interfaz;
- 3) Guia Taller 3 VideoClub para el procedimiento pedagogico;
- 4) ZIP legado solo como referencia visual y de flujos.

Perfil obligatorio del taller: primera fase SQLite, segunda fase MySQL, Bootstrap 5.3 como framework de interfaz, al menos tres apps con CRUD completo y una mejora adicional real. No uses PostgreSQL en esta entrega. No generes todo el proyecto de una vez. Trabaja solo la tarea solicitada, revisa los archivos reales adjuntos y entrega: diagnostico, archivos exactos, codigo completo solo de esos archivos, comandos, pruebas y puerta de salida.

Presenta: (1) una propuesta de alcance para el sistema, (2) una descripcion del desarrollo minimo aceptable para aprobar, (3) funciones base y mejora adicional, (4) lo que debe quedar fuera por tiempo. Debe incluir SQLite primero, MySQL despues, tres CRUD, Bootstrap, login/roles y evidencia. Distingue claramente minimo evaluable de ampliaciones futuras. No generes modelos ni codigo.

ARCHIVOS ESPERADOS

- docs/ALCANCE_PROPUESTO_IA.md

DESPUES DE RECIBIR LA RESPUESTA

1. Guardar la propuesta sin copiarla aun al informe.
2. Subrayar lo util, exagerado e innecesario.
3. Pasar a P-02 para estructurar la apreciacion personal.

EVIDENCIA

- [] Propuesta con alcance base y exclusiones.

NO AVANZAR SI

No diferencia minimo de proyecto completo o incluye funciones empresariales fuera del taller.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continue con P-02. No envie dos prompts de implementacion al mismo tiempo.

P-02

Guia para apreciacion personal

Fase / uso

2.2 del enunciado - Ayudar a estructurar, sin sustituir, el analisis del estudiante.

ANTES DE ENVIAR EL PROMPT

- [] Leer completamente la respuesta P-01.
- [] Anotar al menos tres observaciones propias.
- [] No pedir a la IA que escriba una opinion falsa en primera persona.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Con base en la propuesta anterior, crea una plantilla de analisis critico con tres apartados: "correcto o util", "exagerado o innecesario" y "ajustes que realizaria". Formula preguntas guia y una tabla para que yo la complete. No escribas mi apreciacion personal ni afirmes que comprendo algo que no he explicado.

ARCHIVOS ESPERADOS

- docs/PLANTILLA_APRECIACION.md
- docs/APRECIACION_PERSONAL.md redactado por el estudiante

DESPUES DE RECIBIR LA RESPUESTA

1. Completar la plantilla con palabras propias.
2. Mencionar por que se priorizan tres CRUD, Bootstrap, login y migracion MySQL.
3. Agregar nombre y fecha.

EVIDENCIA

- [] Apreciacion personal original.

NO AVANZAR SI

El texto queda como una respuesta generica de IA o no contiene ajustes personales.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-03. No envíe dos prompts de implementacion al mismo tiempo.

P-03

Modelo entidad-relacion

Fase / uso

2.3 del enunciado - Definir entidades y relaciones coherentes para cinco apps.

ANTES DE ENVIAR EL PROMPT

- [] P-01/P-02 terminados.
- [] Adjuntar reglas maestras y plan tecnico.
- [] Tener claro que el modelo debe ser portable SQLite/MySQL.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Estoy desarrollando el Taller #3 de "Loteria Binaria" con Django. Usa esta autoridad:

- 1) enunciado del Taller #3 para forma de entrega;
- 2) cinco documentos canonicos del MVP Django para reglas de negocio e interfaz;
- 3) Guia Taller 3 VideoClub para el procedimiento pedagogico;
- 4) ZIP legado solo como referencia visual y de flujos.

Perfil obligatorio del taller: primera fase SQLite, segunda fase MySQL, Bootstrap 5.3 como framework de interfaz, al menos tres apps con CRUD completo y una mejora adicional real. No uses PostgreSQL en esta entrega. No generes todo el proyecto de una vez. Trabaja solo la tarea solicitada, revisa los archivos reales adjuntos y entrega: diagnostico, archivos exactos, codigo completo solo de esos archivos, comandos, pruebas y puerta de salida.

Genera el modelo entidad-relacion del alcance del Taller #3. Incluye como minimo accounts, vendors y lottery, y conserva core y finance por coherencia. Modelos base: User personalizado, TermsVersion/Acceptance, AuditEvent, Wallet, Movement, VendorProfile, ConversionRequest/Assignment, LotteryProduct, DrawEvent, Ticket y DrawResult. Para cada entidad indica PK, campos, FK, cardinalidad, obligatoriedad, unicidad, estados y reglas de eliminacion. Montos en enteros minor units. Evita tipos o constraints exclusivos de PostgreSQL. Entrega Mermaid ER y una explicacion. No generes codigo Django todavia.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufixo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- docs/MODELO_ER.md
- diagrama Mermaid o imagen exportada

DESPUES DE RECIBIR LA RESPUESTA

1. Revisar cardinalidades.
2. Confirmar que User se define antes de migrar.
3. Exportar el diagrama para el Word.

EVIDENCIA

[] ER legible con al menos tres modulos.

NO AVANZAR SI

Faltan cardinalidades, usa una app por tabla o introduce modelos no aprobados.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-04. No envíe dos prompts de implementacion al mismo tiempo.

P-04

DDL MySQL e inserts de validacion

Fase / uso

Guia docente 5.1 - Validar el ER sin crear manualmente la base final.

ANTES DE ENVIAR EL PROMPT

- [] P-03 aprobado.
- [] Crear una base temporal llamada loteria_taller3_validacion.
- [] No usar la base final.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Usa el modelo ER aprobado para generar dos scripts MySQL 8: (A) DDL completo para una base TEMPORAL de validacion y (B) INSERT con al menos 10 registros coherentes por tabla cuando sea razonable. Incluye orden de ejecucion, claves foraneas, checks portables y consultas de verificacion. No incluyas credenciales reales. Aclara que esta base se elimina despues y que la base final sera creada por migraciones Django.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- docs/sql/00_validacion_ddl.sql
- docs/sql/01_validacion_inserts.sql
- docs/sql/02_validacion_checks.sql

DESPUES DE RECIBIR LA RESPUESTA

1. Revisar SQL antes de ejecutar.
2. Ejecutar solo en loteria_taller3_validacion.
3. Verificar datos y luego DROP DATABASE loteria_taller3_validacion.
4. Tomar captura sin contraseñas.

EVIDENCIA

- [] Workbench con tablas/datos temporales.
- [] Captura de DROP o nota de eliminacion.

NO AVANZAR SI

El script se pretende usar como base final o crea tablas que no existen en el ER.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-05. No envíe dos prompts de implementación al mismo tiempo.

P-05

Lista de apps por modulo

Fase / uso

2.4 del enunciado - Confirmar responsabilidades y dependencias.

ANTES DE ENVIAR EL PROMPT

[] ER aprobado.

[] No crear app por cada tabla.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

En base al ER y las reglas aprobadas, presenta la lista de aplicaciones Django por modulo. Deben ser accounts, core, finance, vendors y lottery. Para cada app indica modelos, vistas principales, templates, dependencias permitidas y lo que no debe contener. Identifica cuales tres apps demostraran CRUD completo para la rubrica y como se manejara la eliminacion protegida.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- docs/PLAN_APPS.md

DESPUES DE RECIBIR LA RESPUESTA

1. Comparar con la estructura canonica.
2. Aprobar CRUD en accounts, vendors y lottery.
3. No crear todavia las apps si el plan no esta cerrado.

EVIDENCIA

[] Tabla app -> modelos -> CRUD.

NO AVANZAR SI

Propone una app por tabla o mezcla finanzas dentro de views de lottery.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continue con P-06. No envíe dos prompts de implementacion al mismo tiempo.

P-06

Models.py consolidado de referencia

Fase / uso

2.4/2.5 del enunciado - Generar el modelo completo para revision antes de dividirlo.

ANTES DE ENVIAR EL PROMPT

- [] ER y apps aprobados.
- [] Recordar que el archivo no se ejecutara como modelo unico.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Estoy desarrollando el Taller #3 de "Loteria Binaria" con Django. Usa esta autoridad:

- 1) enunciado del Taller #3 para forma de entrega;
- 2) cinco documentos canonicos del MVP Django para reglas de negocio e interfaz;
- 3) Guia Taller 3 VideoClub para el procedimiento pedagogico;
- 4) ZIP legado solo como referencia visual y de flujos.

Perfil obligatorio del taller: primera fase SQLite, segunda fase MySQL, Bootstrap 5.3 como framework de interfaz, al menos tres apps con CRUD completo y una mejora adicional real. No uses PostgreSQL en esta entrega. No generes todo el proyecto de una vez. Trabaja solo la tarea solicitada, revisa los archivos reales adjuntos y entrega: diagnostico, archivos exactos, codigo completo solo de esos archivos, comandos, pruebas y puerta de salida.

Genera un models.py CONSOLIDADO DE REFERENCIA que represente el ER aprobado. Condiciones: User personalizado basado en AbstractUser; BigAutoField/UUID segun el plan; TextChoices; montos BigIntegerField *_minor; ForeignKey con PROTECT para historia; __str__, Meta, indexes, UniqueConstraint/CheckConstraint portables a SQLite/MySQL; sin señales; sin datos; sin vistas/forms/admin. Marca con comentarios a que app pertenece cada modelo. Evita constraints condicionales o campos exclusivos de PostgreSQL. Al final explica como dividirlo. No generes migraciones.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- docs/modelo_consolidado_referencia.py

DESPUES DE RECIBIR LA RESPUESTA

1. Guardar el archivo solo en docs/.
2. Revisar cada campo contra el ER.
3. No importarlo en INSTALLED_APPS.
4. Usarlo como fuente para prompts por app.

EVIDENCIA

[] Archivo consolidado revisado.

NO AVANZAR SI

La IA indica copiarlo completo a una sola app, usa float o genera señales/operaciones automaticas en save().

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-07. No envíe dos prompts de implementacion al mismo tiempo.

P-07

Plan paso a paso por app

Fase / uso

2.5 del enunciado - Planificar la programacion sin generar todo junto.

ANTES DE ENVIAR EL PROMPT

[] P-06 aprobado.

[] Tener la rubrica a la vista.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Planifica la implementacion de accounts, core, finance, vendors y lottery en orden. Para cada app separa: models, migrations, admin, forms, views CRUD/acciones, urls, templates, tests y evidencia. Indica dependencias y comando de salida. Prioriza User antes de migrate, SQLite, tres CRUD, Bootstrap, login/roles y luego MySQL. No generes codigo; produce una secuencia de tareas con puertas de salida.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- docs/PLAN_IMPLEMENTACION_APPS.md

DESPUES DE RECIBIR LA RESPUESTA

1. Convertir el plan en checklist.
2. No iniciar una app si depende de un modelo no migrado.
3. Usar P-09 en adelante para ejecutar.

EVIDENCIA

[] Plan con orden y puertas de salida.

NO AVANZAR SI

El plan empieza migrando antes del User o intenta implementar todos los flujos a la vez.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-08. No envíe dos prompts de implementacion al mismo tiempo.

P-08

Auditoria de reglas antiguas e interfaz

Fase / uso

Antes de implementar UI - Clasificar el ZIP sin reintroducir contradicciones.

ANTES DE ENVIAR EL PROMPT

- [] Adjuntar ZIP legado.
- [] Adjuntar auditoria canonica.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Audita el ZIP legado pantalla por pantalla y regla por regla. Entrega: mapa de HTML/CSS/JS/JSON; funciones visibles; reglas heredadas; tabla CONSERVAR/ADAPTAR/RETIRAR; mapeo a templates, views, forms y modelos Django; enlaces o funciones muertas. Debes retirar 5% de recarga, 15% de conversion, 75% de premio, cliente financiero limitado, localStorage autoritativo, passwords JSON, Hex 4 y reclamo manual. Conserva identidad azul/dorado, filtros de boletos, dashboards y detalles. No escribas codigo.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- docs/AUDITORIA_ZIP_TALLER3.md
- docs/MAPA_INTERFAZ.md

DESPUES DE RECIBIR LA RESPUESTA

1. Revisar que todas las pantallas tengan destino Django.
2. Marcar funciones futuras para no mostrar botones muertos.
3. Usar el mapa en P-17 a P-21.

EVIDENCIA

- [] Tabla de reglas y mapa de pantallas.

NO AVANZAR SI

Se conservan reglas supersedidas o se afirma que JSON/localStorage sigue siendo base de datos.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continue con P-09. No envíe dos prompts de implementacion al mismo tiempo.

P-09

Auditoria del repositorio actual

Fase / uso

Antes de comandos - Evitar recrear archivos o migraciones existentes.

ANTES DE ENVIAR EL PROMPT

[] Adjuntar el repositorio/ZIP actual, no solo el legado.

[] Ejecutar tree o listar carpetas.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Estoy desarrollando el Taller #3 de "Loteria Binaria" con Django. Usa esta autoridad:

- 1) enunciado del Taller #3 para forma de entrega;
- 2) cinco documentos canonicos del MVP Django para reglas de negocio e interfaz;
- 3) Guia Taller 3 VideoClub para el procedimiento pedagogico;
- 4) ZIP legado solo como referencia visual y de flujos.

Perfil obligatorio del taller: primera fase SQLite, segunda fase MySQL, Bootstrap 5.3 como framework de interfaz, al menos tres apps con CRUD completo y una mejora adicional real. No uses PostgreSQL en esta entrega. No generes todo el proyecto de una vez. Trabaja solo la tarea solicitada, revisa los archivos reales adjuntos y entrega: diagnostico, archivos exactos, codigo completo solo de esos archivos, comandos, pruebas y puerta de salida.

Audita el repositorio actual sin modificarlo. Revisa manage.py, config/settings.py, urls.py, apps, AUTH_USER_MODEL, migrations, templates, static, requirements, .gitignore y bases existentes. Indica si se parte de cero o se continua. Entrega tabla existe/incompleto/falta/contradice, migraciones que no deben editarse, comandos de diagnostico y un siguiente paso unico.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- docs/AUDITORIA_REPOSITORIO.md

DESPUES DE RECIBIR LA RESPUESTA

1. Ejecutar los comandos propuestos.
2. Pegar salidas de error en el mismo chat.
3. No borrar db.sqlite3 ni migraciones sin diagnostico.

COMANDOS DE VALIDACION

```
python --version
python -m django --version
python manage.py check
```

```
python manage.py showmigrations
```

EVIDENCIA

[] Informe y salidas de consola.

NO AVANZAR SI

La IA asume archivos no inspeccionados o recomienda borrar la base/migrations.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-10. No envíe dos prompts de implementación al mismo tiempo.

FASE 1

Entorno virtual, proyecto y configuracion SQLite

Crear la base Django sin ejecutar la primera migracion antes del User.

Resultado que debe existir al terminar

Entorno .venv, requirements, proyecto config, cinco apps, settings y SQLite configurada; aun sin migrate.

Secuencia intercalada de esta fase

Paso	Accion del estudiante	Prompt / archivo	Comprobacion para seguir
1.1	Comprobar Python, pip, Git y MySQL futuro.	P-10	Versiones registradas.
1.2	Crear y activar .venv; instalar Django.	P-11	(.venv) visible y requirements.txt.
1.3	Crear config y cinco apps.	P-12	manage.py check sin migrate.
1.4	Configurar settings, templates, static y SQLite.	P-13	AUTH_USER_MODEL aun pendiente; no migrate.

1.0 - Ubicacion correcta antes de crear el proyecto

1. Abre PowerShell en la carpeta vacia o preparada para el Taller #3.
2. No ejecutes django-admin startproject dentro de otra carpeta config existente.
3. Comprueba que el nombre de la ruta no sea una copia accidental del proyecto VideoClub.

Comandos

```
Get-Location
Get-ChildItem
```

Evidencia que debes guardar

[] Captura de la ruta de trabajo.

P-10

Prerequisitos de entorno

Fase / uso

Fase entorno - Confirmar herramientas antes de crear .venv.

ANTES DE ENVIAR EL PROMPT

- [] Abrir PowerShell.
- [] Estar en la carpeta padre del proyecto.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Dame un checklist de diagnostico para Windows PowerShell de Python 3.12, pip, Git y MySQL, diferenciando lo necesario para SQLite y lo que se instalara despues para MySQL. Incluye comandos y como interpretar cada salida. No instales ni generes proyecto todavia.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.

- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- evidencias/01_entorno/diagnostico.txt

DESPUES DE RECIBIR LA RESPUESTA

1. Ejecutar uno por uno.
2. Guardar salidas.
3. Resolver Python antes de seguir.

COMANDOS DE VALIDACION

```
py -0p
py -3.12 --version
git --version
mysql --version
```

EVIDENCIA

- [] Pantalla completa con versiones.

NO AVANZAR SI

Python 3.12 no esta disponible o el comando python apunta a otra instalacion desconocida.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-11. No envíe dos prompts de implementacion al mismo tiempo.

P-11

Crear .venv e instalar Django

Fase / uso

Fase 1 - Construir el entorno aislado y reproducible.

ANTES DE ENVIAR EL PROMPT

[] P-10 verde.

[] No hay .venv antigua incompatible.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Indica exactamente como crear y activar .venv en Windows PowerShell, instalar Django 5.2 y django-environ, generar requirements.txt y configurar .gitignore. Incluye solucion para ExecutionPolicy solo a nivel Process. Explica que .venv se activa una vez por terminal y se desactiva al finalizar. No crees el proyecto aun.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- .venv/
- requirements.txt
- .gitignore

DESPUES DE RECIBIR LA RESPUESTA

1. Ejecutar comandos exactamente.
2. Verificar que el prompt muestra (.venv).
3. Capturar version.

COMANDOS DE VALIDACION

```
py -3.12 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install "Django~=5.2" django-environ
pip freeze > requirements.txt
python -m django --version
```

EVIDENCIA

[] .venv activo y version Django.

NO AVANZAR SI

pip instala fuera de .venv o se intenta subir .venv a Git.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-12. No envíe dos prompts de implementación al mismo tiempo.

P-12

Crear proyecto y apps

Fase / uso

Fase 2 - Crear estructura sin migrar.

ANTES DE ENVIAR EL PROMPT

- [] .env activo.
- [] Carpeta sin manage.py.
- [] P-05 aprobado.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Guia la creacion del proyecto Django config en la carpeta actual y las apps dentro de apps/: accounts, core, finance, vendors y lottery. Proporciona comandos PowerShell, contenido correcto de cada apps.py con name="apps.<app>", INSTALLED_APPS y arbol esperado. No ejecutes migrate. Indica si manage.py debe editarse (normalmente no).

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- manage.py
- config/
- apps/<cinco apps>

DESPUES DE RECIBIR LA RESPUESTA

1. Ejecutar comandos.
2. Editar apps.py e INSTALLED_APPS.
3. Ejecutar check.

COMANDOS DE VALIDACION

```
django-admin startproject config .
mkdir apps
New-Item apps\__init__.py -ItemType File
python manage.py startapp accounts apps/accounts
python manage.py startapp core apps/core
python manage.py startapp finance apps/finance
python manage.py startapp vendors apps/vendors
python manage.py startapp lottery apps/lottery
python manage.py check
```

EVIDENCIA

[] Arbol y check verde.

NO AVANZAR SI

Se ejecuto migrate o una app tiene name incorrecto.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continue con P-13. No envíe dos prompts de implementacion al mismo tiempo.

P-13

Configurar settings y SQLite

Fase / uso

Fase 3 - Preparar templates/static/env y base SQLite.

ANTES DE ENVIAR EL PROMPT

- [] Proyecto creado.
- [] .venv activo.
- [] No existe primera migracion del proyecto.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Revisa y genera los cambios exactos de config/settings.py para: django-environ, .env, DATABASE_URL con SQLite por defecto, TIME_ZONE="America/Guayaquil", USE_TZ=True, templates globales, static, LOGIN_URL y cinco apps. Crea .env.example. Explica que manage.py se usa, no se edita para cambiar la base. No ejecutes migrate. Entrega el archivo completo de settings.py.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- config/settings.py
- .env
- .env.example
- templates/
- static/

DESPUES DE RECIBIR LA RESPUESTA

1. Crear respaldos.
2. Pegar cambios.
3. Comprobar que .env esta ignorado.
4. Ejecutar check.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py diffsettings
```

EVIDENCIA

[] settings y .env.example.

NO AVANZAR SI

DATABASE_URL apunta a MySQL antes de la fase SQLite o .env queda versionado.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-14. No envíe dos prompts de implementación al mismo tiempo.

1.5 - Puerta de salida de la fase

1. Abre cada apps.py y confirma name="apps.accounts", name="apps.core", etc.
2. Abre config/settings.py y confirma las cinco apps, templates globales, static y SQLite.
3. No ejecutes migrate hasta terminar P-14 y configurar AUTH_USER_MODEL.

Comandos

```
python manage.py check
```

Evidencia que debes guardar

[] Salida System check identified no issues.

**NO PROSIGAS
TODAVIA**

Ya ejecutaste migrate sin User personalizado, hay apps con name incorrecto o settings no carga.

FASE 2

Usuario personalizado, primera migracion y administracion

Definir User antes de migrar y dejar SQLite funcional.

Resultado que debe existir al terminar

accounts.User definitivo, migraciones aplicadas, superusuario, admin seguro y seed idempotente.

Secuencia intercalada de esta fase

Paso	Accion del estudiante	Prompt / archivo	Comprobacion para seguir
2.1	Crear User, terminos y AUTH_USER_MODEL.	P-14	Migracion accounts revisada antes de migrate.
2.2	Aplicar primera migracion SQLite y crear superusuario.	P-15	/admin/ accesible.
2.3	Registrar modelos y crear seed demo.	P-16	Segunda ejecucion sin duplicados.

P-14

User personalizado y primera migracion

Fase / uso

Fase 4 - Definir identidad antes de cualquier migrate.

ANTES DE ENVIAR EL PROMPT

- [] No existe db.sqlite3.
- [] AUTH_USER_MODEL aun no migrado.
- [] P-06 disponible.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Genera solo apps/accounts/models.py con User basado en AbstractUser y modelos TermsVersion/TermsAcceptance segun la baseline. Campos User: email unico normalizado, document unico, phone, birth_date, status, created_at, updated_at. Agrega validaciones estructurales portables SQLite/MySQL, __str__, Meta e indices. Indica cambio AUTH_USER_MODEL y admin minimo. No generes otras apps ni signals.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/accounts/models.py
- apps/accounts/admin.py
- config/settings.py
- apps/accounts/migrations/0001_initial.py

DESPUES DE RECIBIR LA RESPUESTA

1. Revisar codigo.

2. Configurar AUTH_USER_MODEL.
3. Ejecutar makemigrations accounts.
4. Abrir y revisar 0001.
5. Ejecutar sqlmigrate.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py makemigrations accounts
python manage.py sqlmigrate accounts 0001
```

EVIDENCIA

[] Migration 0001 revisada.

NO AVANZAR SI

Ya existe una primera migracion aplicada con auth.User o faltan unicidad/estado.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-15. No envíe dos prompts de implementacion al mismo tiempo.

P-15

Migrar SQLite y crear superusuario

Fase / uso

Fase 5 - Crear la primera base funcional.

ANTES DE ENVIAR EL PROMPT

[] P-14 verde.

[] DATABASE_URL SQLite.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Dame el orden exacto para aplicar migraciones en SQLite, comprobar la base, crear superusuario y validar /admin/. Incluye comandos de showmigrations, shell para verificar AUTH_USER_MODEL y pasos de captura. No propongas borrar migraciones si falla; pide la salida exacta.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- db.sqlite3
- superusuario local

DESPUES DE RECIBIR LA RESPUESTA

1. Ejecutar migrate.
2. Crear superusuario.
3. Iniciar servidor y abrir /admin/.
4. Detener con Ctrl+C.

COMANDOS DE VALIDACION

```
python manage.py migrate
python manage.py showmigrations
python manage.py shell -c "from django.contrib.auth import get_user_model; print(get_user_model())"
python manage.py createsuperuser
python manage.py runserver
```

EVIDENCIA

[] Migraciones y admin.

NO AVANZAR SI

AUTH_USER_MODEL no es accounts.User o el admin no autentica.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-16. No envíe dos prompts de implementación al mismo tiempo.

P-16

Admin seguro y seed demo

Fase / uso

Fases 6-7 - Administrar datos y crear demostracion reproducible.

ANTES DE ENVIAR EL PROMPT

[] Modelos por apps migrados o listos.

[] No usar datos secretos.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Genera ModelAdmin detallados y un management command seed_demo idempotente para los modelos existentes. Usa update_or_create, set_password solo para usuarios demo y fechas relativas. Movement, Ticket, DrawResult, TermsAcceptance y AuditEvent deben ser de solo lectura o sin delete. Entrega un app por vez si el codigo es largo; empieza por accounts/core y espera confirmacion.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/*/admin.py
- apps/core/management/commands/seed_demo.py

DESPUES DE RECIBIR LA RESPUESTA

1. Aplicar solo el bloque entregado.
2. Ejecutar check y seed dos veces.
3. Comparar conteos.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py seed_demo
python manage.py seed_demo
```

EVIDENCIA

[] Admin con filtros y seed sin duplicados.

NO AVANZAR SI

El seed duplica, guarda password directo o permite editar saldos historicos.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-17. No envíe dos prompts de implementación al mismo tiempo.

2.4 - Comprobacion manual en SQLite

1. Inicia el servidor y entra a /admin/ con el superusuario.
2. Comprueba que aparece Accounts y que User usa los campos personalizados.
3. Crea un usuario de prueba desde admin y confirma que la contraseña no se muestra en texto.
4. Cierra el servidor con Ctrl+C antes de seguir editando migraciones.

Comandos

```
python manage.py runserver
```

Evidencia que debes guardar

- [] Admin funcional.
- [] Tabla django_migrations o showmigrations.
- [] Usuario creado con password hash.

**NO PROSIGAS
TODAVIA**

No puedes entrar al admin, AUTH_USER_MODEL no es accounts.User o la base contiene tablas de un User anterior.

FASE 3

Base visual, paginas y migracion del frontend

Convertir la interfaz antigua en templates Django con Bootstrap.

Resultado que debe existir al terminar

base.html, app.css, paginas publicas, selector, dashboards, responsive y mapa de migracion sin JSON/localStorage de negocio.

Secuencia intercalada de esta fase

Paso	Accion del estudiante	Prompt / archivo	Comprobacion para seguir
3.1	Crear base.html y estilos Bootstrap.	P-17	Navbar, messages, footer y blocks funcionan.
3.2	Crear landing, login y registro visual.	P-18	Sin reglas antiguas ni enlaces muertos.
3.3	Crear selector y dashboards por modo.	P-19	Navegacion aislada por contexto.
3.4	Auditar responsive y accesibilidad.	P-20	Pruebas 360-1440 px.
3.5	Migrar CSS/JS/JSON archivo por archivo.	P-21	ZIP de respaldo intacto.

P-17

BASE.HTML con Bootstrap 5.3

Fase / uso

2.6 del enunciado - Crear la base visual profesional.

ANTES DE ENVIAR EL PROMPT

- [] Static/templates configurados.
- [] Bootstrap elegido como unico framework.
- [] Mapa visual P-08 aprobado.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Genera templates/base.html para Loteria Binaria usando Bootstrap 5.3: HTML5, viewport, navbar responsive, logo, enlaces calculados por backend, messages accesibles, block title/content/extra_css/extra_js, footer academico y bootstrap.bundle.js. Crea static/css/app.css con paleta azul oscuro/dorado y ajustes minimos. Usa clases Bootstrap reales; no recrees Bootstrap en CSS. Entrega ambos archivos completos y explica donde guardarlos.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y disenio responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

ARCHIVOS ESPERADOS

- templates/base.html
- templates/includes/_messages.html
- static/css/app.css

DESPUES DE RECIBIR LA RESPUESTA

1. Crear carpetas.
2. Pegar archivos.
3. Crear una vista temporal o home.
4. Abrir escritorio y movil.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py runserver
```

EVIDENCIA

[] Navbar, cards y responsive visibles.

NO AVANZAR SI

No carga Bootstrap, hay 404 static o la pagina usa solo CSS propio.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-18. No envíe dos prompts de implementación al mismo tiempo.

P-18

Landing, login y registro visual

Fase / uso

Fase interfaz publica - Transformar paginas legadas en templates Django.

ANTES DE ENVIAR EL PROMPT

- [] P-17 funciona.
- [] No implementar auth completa aun si models/forms no estan listos.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Usa el ZIP legado y el mapa P-08 para generar solo los templates publicos: core/home.html, accounts/login.html y accounts/register.html extendiendo base.html. Conserva identidad, explicacion Octal/Decimal/Hex, naturaleza academica y formularios Bootstrap. Retira porcentajes 5/15/75, passwords demo visibles y enlaces muertos. Usa {% url %}, {% static %}, csrf_token y renderizado de errores. No generes views aun.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y disenio responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

ARCHIVOS ESPERADOS

- templates/core/home.html
- templates/accounts/login.html
- templates/accounts/register.html

DESPUES DE RECIBIR LA RESPUESTA

1. Guardar templates.
2. Revisar etiquetas y textos.
3. No dejar href a pages/*.html.

COMANDOS DE VALIDACION

```
python manage.py check
```

EVIDENCIA

- [] Templates sin rutas legadas.

NO AVANZAR SI

Mantiene reglas antiguas o formularios sin CSRF.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-19. No envíe dos prompts de implementación al mismo tiempo.

P-19

Dashboards y selector de modo

Fase / uso

Fase interfaz privada - Recrear Cliente, Vendedor y Administrador.

ANTES DE ENVIAR EL PROMPT

[] P-17/P-18 listos.

[] Roles y modo definidos en documentos.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Genera templates/accounts/choose_mode.html y templates/dashboards/client.html, vendor.html, admin.html. Todos extienden base.html o base_dashboard.html, usan Bootstrap y reciben contextos de Django. El selector solo muestra roles asignados. Modo CLIENTE tiene funciones completas; VENDEDOR/ADMINISTRADOR no compran. Incluye estados vacios, filtros visuales de boletos y aviso academico. No uses localStorage para rol/saldo.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

ARCHIVOS ESPERADOS

- templates/accounts/choose_mode.html
- templates/dashboards/client.html
- vendor.html
- admin.html

DESPUES DE RECIBIR LA RESPUESTA

1. Crear templates.
2. Validar que no hay acciones contradictorias.
3. Preparar contextos requeridos para P-31/P-32.

COMANDOS DE VALIDACION

```
python manage.py check
```

EVIDENCIA

[] Cuatro pantallas consistentes.

NO AVANZAR SI

El modo se decide con JS/localStorage o se muestra "Cliente financiero".

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-20. No envíe dos prompts de implementación al mismo tiempo.

P-20

Responsive y accesibilidad Bootstrap

Fase / uso

Fase UI - Comprobar calidad visual y requisito de framework.

ANTES DE ENVIAR EL PROMPT

[] Templates base y dashboards creados.

[] DevTools disponible.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Audita los templates y app.css para 360, 390, 768, 1024 y 1440 px. Corrige usando grid, navbar collapse/offcanvas, table-responsive, cards y utilities Bootstrap. Revisa H1 unico, labels, foco, contraste, aria-live, objetivos tactiles y que la barra no tape contenido. Entrega solo cambios necesarios por archivo y una matriz de prueba manual.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

ARCHIVOS ESPERADOS

- static/css/app.css
- templates afectados
- docs/PRUEBAS_RESPONSIVE.md

DESPUES DE RECIBIR LA RESPUESTA

1. Aplicar cambios.
2. Probar cada ancho.
3. Guardar capturas completas.

COMANDOS DE VALIDACION

```
python manage.py runserver
```

EVIDENCIA

[] Cinco anchos sin overflow.

NO AVANZAR SI

Hay scroll horizontal global, menu inaccesible o estados dependen solo de color.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-21. No envíe dos prompts de implementación al mismo tiempo.

P-21

Migrar HTML/CSS/JS legado

Fase / uso

Fase UI - Reutilizar sin funciones muertas ni autoridad local.

ANTES DE ENVIAR EL PROMPT

- [] Auditoria P-08.
- [] Templates base creados.
- [] Respaldo del ZIP.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Crea un plan archivo por archivo para migrar el ZIP a templates/static. Para cada HTML indica destino; para cada JS clasifica conservar solo UI, reescribir como POST Django o retirar; para JSON indica seed o descartar. Despues genera cambios de un solo modulo por respuesta. Deben desaparecer localStorage de negocio, fetch JSON, credenciales demo y enlaces pages/*.html. No borres el ZIP de respaldo.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

ARCHIVOS ESPERADOS

- static/js/app.js y modulos visuales necesarios
- static/img/
- templates migrados
- seed_data/legacy opcional

DESPUES DE RECIBIR LA RESPUESTA

1. Migrar un modulo.
2. Ejecutar check/runserver.
3. Buscar referencias a localStorage/fetch data/.

COMANDOS DE VALIDACION

```
rg -n "localStorage|data/*.json|pages/*.html" templates static apps
```

EVIDENCIA

- [] Busqueda sin autoridad legada.

NO AVANZAR SI

Se copian nueve JS completos sin revisar o quedan funciones que modifican saldos en navegador.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-22. No envíe dos prompts de implementación al mismo tiempo.

3.6 - Lista visual que debes comprobar

1. Landing: hero, explicacion de Octal/Decimal/Hexadecimal, como funciona, roles, seguridad, llamada a accion y aviso academico.
2. Login: usuario/correo, contrasena, errores, enlace de registro y sin credenciales demo visibles.
3. Registro: datos personales, mayoria de edad, terminos, password y confirmacion.
4. Selector: tarjetas solo para roles asignados, descripcion real y boton Entrar como...
5. Cliente: saldos, sorteos, boletos, wallet, solicitudes e historial.
6. Vendedor: inventario, solicitudes elegibles, ventas y wallet; sin compra de boletos en modo VENDEDOR.
7. Administrador: usuarios, vendedores, productos/eventos, resultados, movimientos y auditoria.

Evidencia que debes guardar

[] Capturas escritorio y movil de las paginas disponibles.

**NO PROSIGAS
TODAVIA**

Aparece 5%, 15%, 75%, hexadecimal de 4 caracteres, botones sin URL o localStorage como autoridad.

FASE 4

CRUD completo 1 de 3 - Accounts

Construir el primer CRUD siguiendo el ciclo models/forms/admin/views/urls/templates/tests.

Resultado que debe existir al terminar

CRUD administrativo de usuarios y terminos con Bootstrap, permisos, validaciones y desactivacion protegida.

Secuencia intercalada de esta fase

Paso	Accion del estudiante	Prompt / archivo	Comprobacion para seguir
4.1	Revisar modelos existentes y crear forms/admin.	P-22	Forms testados y admin seguro.
4.2	Crear views, urls y templates CRUD.	P-23	C/R/U/D probado por administrador.

P-22

Accounts: models, forms y admin

Fase / uso

CRUD 1/3 - Preparar el primer CRUD evaluable.

ANTES DE ENVIAR EL PROMPT

- [] User ya migrado.
- [] P-06/P-07 disponibles.
- [] Adjuntar archivos actuales.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Revisa accounts actual y genera solo forms.py y admin.py necesarios para CRUD de usuarios y terminos. Validaciones: mayoria de edad, email/documento unicos, password mediante set_password/UserCreationForm, estado y campos permitidos. El admin no borra historia; desactiva cuando corresponda. Entrega codigo completo y tests de forms. No generes views/templates.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/accounts/forms.py
- apps/accounts/admin.py
- apps/accounts/tests/test_forms.py

DESPUES DE RECIBIR LA RESPUESTA

1. Pegar codigo.
2. Ejecutar check y tests de accounts.
3. Probar formulario en shell/admin.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py test apps.accounts
```

EVIDENCIA

[] Forms y admin verdes.

NO AVANZAR SI

El formulario guarda password plano o permite fecha de nacimiento invalida.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-23. No envíe dos prompts de implementación al mismo tiempo.

P-23

Accounts: CRUD views, URLs y templates

Fase / uso

CRUD 1/3 - Completar Create/Read/Update/Delete controlado.

ANTES DE ENVIAR EL PROMPT

[] P-22 verde.

[] base.html funciona.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Genera CRUD completo de accounts.User para administradores: ListView con busqueda/paginacion, DetailView, CreateView, UpdateView y Delete/DeactivateView. La eliminacion fisica solo si no existe historia; de lo contrario is_active=False y status desactivado con mensaje. Protege rutas. Crea urls.py y templates list/form/detail/confirm_delete Bootstrap. Entrega codigo por archivo y tests de acceso/CRUD.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y disenio responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/accounts/views.py
- apps/accounts/urls.py
- templates/accounts/user_*.html
- apps/accounts/tests/test_views.py

DESPUES DE RECIBIR LA RESPUESTA

1. Crear archivos.
2. Incluir URLs en config temporalmente.

3. Ejecutar tests.
4. Probar cinco rutas.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py test apps.accounts
python manage.py runserver
```

EVIDENCIA

[] CRUD accounts completo.

NO AVANZAR SI

Un cliente accede al CRUD o eliminar usuario borra movimientos/boletos.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-24. No envíe dos prompts de implementación al mismo tiempo.

4.3 - Prueba manual del CRUD Accounts

1. Crear usuario adulto y aceptar terminos.
2. Intentar duplicar username, email y documento.
3. Abrir detalle, editar campos permitidos y comprobar mensajes.
4. Intentar eliminar un usuario sin historia y otro con historia; el segundo debe desactivarse o bloquearse.
5. Entrar con una cuenta no administrativa y comprobar 403/redireccion.

Comandos

```
python manage.py test apps.accounts
python manage.py runserver
```

Evidencia que debes guardar

[] Lista, formulario, detalle, edicion, confirmacion y resultado de eliminacion protegida.

**NO PROSIGAS
TODAVIA**

Falta una operacion CRUD, una contrasena se asigna como texto o un usuario comun entra al CRUD administrativo.

FASE 5

CRUD completo 2 de 3 - Vendors

Construir VendorProfile y exponer solicitudes como flujo protegido.

Resultado que debe existir al terminar

CRUD de perfiles vendedores y listado de solicitudes sin editar estados mediante CRUD generico.

Secuencia intercalada de esta fase

Paso	Accion del estudiante	Prompt / archivo	Comprobacion para seguir
5.1	Crear/revisar models, forms y admin.	P-24	Migracion portable SQLite/MySQL.
5.2	Crear views, urls, templates y tests.	P-25	CRUD solo administrador.

P-24

Vendors: models, forms y admin

Fase / uso

CRUD 2/3 - Preparar perfiles y solicitudes portables.

ANTES DE ENVIAR EL PROMPT

[] Accounts CRUD verde.

[] Adjuntar vendors actual.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Genera/revisa vendors/models.py, forms.py y admin.py. VendorProfile es OneToOne con User; ConversionRequest y Assignment usan estados, montos minor, fecha de expiracion y relaciones PROTECT. Para portabilidad SQLite/MySQL evita UniqueConstraint condicional: usa OneToOne o validacion de servicio para una asignacion actual. CRUD evaluable es VendorProfile; solicitudes son flujo. Incluye migracion propuesta y tests de modelo/form.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/vendors/models.py

- forms.py
- admin.py
- migration nueva
- tests

DESPUES DE RECIBIR LA RESPUESTA

1. Revisar migracion.
2. Makemigrations vendors.
3. Migrate SQLite.
4. Ejecutar tests.

COMANDOS DE VALIDACION

```
python manage.py makemigrations vendors
python manage.py sqlmigrate vendors <NUMERO>
python manage.py migrate
python manage.py test apps.vendors
```

EVIDENCIA

[] Modelos/vendors verdes.

NO AVANZAR SI

Hay dos perfiles por usuario, constraints PostgreSQL-only o monto float.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-25. No envíe dos prompts de implementación al mismo tiempo.

P-25

Vendors: CRUD views, URLs y templates

Fase / uso

CRUD 2/3 - Completar perfiles vendedores.

ANTES DE ENVIAR EL PROMPT

[] P-24 migrado.

[] base.html y permisos listos.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Genera CRUD completo Bootstrap para VendorProfile: lista con filtro por estado, detalle, crear, editar y eliminar/desactivar protegido. Solo administrador. Si hay solicitudes/compras, bloquear eliminacion fisica y desactivar. Crea views.py, urls.py, templates y tests. Agrega una vista read-only de solicitudes para demostrar el modulo sin editar estados desde CRUD generico.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/vendors/views.py
- urls.py
- templates/vendors/*
- tests/test_views.py

DESPUES DE RECIBIR LA RESPUESTA

1. Integrar URLs.
2. Ejecutar tests.
3. Probar perfil sin/con historia.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py test apps.vendors
python manage.py runserver
```

EVIDENCIA

[] CRUD vendors y lista solicitudes.

NO AVANZAR SI

Se pueden cambiar estados terminales con ModelForm generico o un vendedor ve solicitudes ajenas no elegibles.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-26. No envíe dos prompts de implementacion al mismo tiempo.

5.3 - Prueba manual del CRUD Vendors

1. Crear VendorProfile solo para un User valido.
2. Intentar crear dos perfiles para el mismo User.
3. Filtrar por estado, abrir detalle y editar.
4. Asociar una solicitud y comprobar que la eliminacion fisica se bloquea o desactiva.
5. Abrir el listado read-only de solicitudes y comprobar que no existe editar estado generico.

Comandos

```
python manage.py test apps.vendors
python manage.py runserver
```

Evidencia que debes guardar

- [] Cinco operaciones del perfil y listado de solicitudes.

**NO PROSIGAS
TODAVIA**

Una solicitud puede editarse libremente desde UpdateView o un perfil con historia se borra.

FASE 6

CRUD completo 3 de 3 - Lottery

Construir productos y eventos con reglas de loteria vigentes.

Resultado que debe existir al terminar

CRUD Bootstrap de LotteryProduct y DrawEvent; Ticket y DrawResult protegidos.

Secuencia intercalada de esta fase

Paso	Accion del estudiante	Prompt / archivo	Comprobacion para seguir
6.1	Crear/revisar models, forms y admin.	P-26	Validaciones 4/5/6 y cierre 10 min.
6.2	Crear views, urls, templates y tests.	P-27	Edicion/borrado segun estado.

P-26

Lottery: models, forms y admin

Fase / uso

CRUD 3/3 - Implementar productos/eventos y reglas esenciales.

ANTES DE ENVIAR EL PROMPT

- [] Vendors CRUD verde.
- [] Adjuntar lottery actual y reglas canonicas.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Genera/revisa lottery/models.py, forms.py y admin.py para LotteryProduct, DrawEvent, Ticket y DrawResult. Productos: OCTAL 0-7/4 unicos, DECIMAL 0-9/5, HEXADECIMAL 0-9A-F/6. DrawEvent cierre=draw_at-10min, premio fijo, estados. Ticket unique(event, normalized_key). DrawResult OneToOne e inmutable. CRUD evaluable: Product y Event; Ticket/Result read-only o acciones. Compatible SQLite/MySQL. Incluye tests de validacion.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/lottery/models.py

- forms.py
- admin.py
- migrations
- tests

DESPUES DE RECIBIR LA RESPUESTA

1. Revisar migracion.
2. Migrate SQLite.
3. Ejecutar tests de productos/eventos.

COMANDOS DE VALIDACION

```
python manage.py makemigrations lottery
python manage.py migrate
python manage.py test apps.lottery
```

EVIDENCIA

- [] Reglas de productos probadas.

NO AVANZAR SI

Hex tiene 4, acepta repetidos o se usa premio 75%.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-27. No envíe dos prompts de implementacion al mismo tiempo.

P-27

Lottery: CRUD views, URLs y templates

Fase / uso

CRUD 3/3 - Completar productos y eventos.

ANTES DE ENVIAR EL PROMPT

[] P-26 verde.

[] base.html lista.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Genera CRUD Bootstrap para LotteryProduct y DrawEvent: listas con filtros/paginacion, detalle, crear, editar y eliminar protegido. Producto con eventos no se elimina. Evento solo se elimina en BORRADOR sin tickets/resultados. Evento publicado no edita producto, precio, premio ni fechas. Crea views.py, urls.py, templates y tests de permisos/reglas. No incluyas compra de boleto todavia.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/lottery/views.py
- urls.py
- templates/lottery/product_*
- event_*
- tests/test_crud.py

DESPUES DE RECIBIR LA RESPUESTA

1. Integrar URLs.
2. Ejecutar tests.

3. Probar eliminacion permitida/bloqueada.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py test apps.lottery
python manage.py runserver
```

EVIDENCIA

[] Tercer CRUD completo.

NO AVANZAR SI

Se puede editar un evento publicado o borrar un producto con eventos.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-28. No envíe dos prompts de implementacion al mismo tiempo.

6.3 - Prueba manual del CRUD Lottery

1. Crear los productos Octal, Decimal y Hexadecimal con sus universos y cardinalidades.
2. Crear un evento borrador con cierre exactamente diez minutos antes del sorteo.
3. Editar el evento mientras es borrador.
4. Publicar o simular estado publicado y comprobar que precio, premio, producto y fechas quedan bloqueados.
5. Intentar borrar un producto con eventos y un evento con tickets/resultados.

Comandos

```
python manage.py test apps.lottery
python manage.py runserver
```

Evidencia que debes guardar

[] Listas, filtros, formularios, detalle y bloqueos.

**NO PROSIGAS
TODAVIA**

Se permite Hexadecimal de 4, simbolos repetidos, cierre variable o edicion de un evento publicado.

FASE 7

Integracion, modulos de apoyo y auditoria de CRUD

Conectar todo sin introducir CRUD destructivo de saldos o historia.

Resultado que debe existir al terminar

Finance/core seguros, URLs integradas, navegacion por rol y matriz de los tres CRUD completa.

Secuencia intercalada de esta fase

Paso	Accion del estudiante	Prompt / archivo	Comprobacion para seguir
7.1	Crear consultas seguras de finance/core.	P-28	Wallet/Movement/AuditEvent read-only.
7.2	Integrar URLs y navegacion.	P-29	Sin 404, NoReverseMatch ni enlaces muertos.
7.3	Auditar los tres CRUD.	P-30	Matriz 3x5 y correcciones cerradas.

P-28

Finance y core sin CRUD destructivo

Fase / uso

Modulos de apoyo - Mostrar datos y operaciones seguras.

ANTES DE ENVIAR EL PROMPT

[] Tres CRUD terminados.

[] Modelos finance/core migrados.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Genera vistas, urls y templates Bootstrap para: wallet propia, movimientos paginados, auditoria administrativa y dashboard home. Wallet/Movement/AuditEvent son read-only en formularios genericos. Cualquier recarga/conversion simulada debe ser una accion POST en services.py, no UpdateView de saldo. Empieza solo por list/detail y entrega tests de propiedad/permisos.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y disenio responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/finance/views.py
- urls.py
- templates/finance/*
- apps/core/views.py
- urls.py
- templates/core/*
- tests

DESPUES DE RECIBIR LA RESPUESTA

1. Aplicar modulo por modulo.
2. Ejecutar tests.
3. Verificar que cambiar pk no expone datos ajenos.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py test apps.finance apps.core
```

EVIDENCIA

[] Listados seguros.

NO AVANZAR SI

Existe un formulario generico para editar balance o borrar movimientos/auditoria.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-29. No envíe dos prompts de implementación al mismo tiempo.

P-29

Integrar URLs y navegacion

Fase / uso

Integracion - Conectar todo sin enlaces muertos.

ANTES DE ENVIAR EL PROMPT

[] URLs de apps creadas.

[] Templates base.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Revisa config/urls.py y todos los app urls.py. Genera el inventario de rutas y el codigo para incluir namespaces accounts, core, finance, vendors y lottery. Actualiza navbar/sidebar para mostrar enlaces segun usuario y modo calculados en backend. Crea redirecciones de inicio apropiadas. No uses rutas relativas a pages/*.html.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- config/urls.py
- apps/*/urls.py
- templates/base.html o includes nav

DESPUES DE RECIBIR LA RESPUESTA

1. Aplicar cambios.
2. Ejecutar check.
3. Recorrer todas las rutas.
4. Buscar NoReverseMatch y 404.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py runserver
```

EVIDENCIA

[] Inventario de rutas y navegacion.

NO AVANZAR SI

Hay nombres duplicados, enlaces 404 o opciones de otro rol.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-30. No envíe dos prompts de implementacion al mismo tiempo.

P-30

Auditoria de los tres CRUD

Fase / uso

Puerta de rubrica - Comprobar C/R/U/D, validaciones y framework.

ANTES DE ENVIAR EL PROMPT

[] P-23, P-25 y P-27 terminados.

[] Servidor en SQLite.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Audita los CRUD de accounts, vendors y lottery usando archivos reales. Crea una matriz por app con ruta de lista/detalle/crear/editar/eliminar, formulario, validaciones, permisos, template Bootstrap, mensaje y prueba automatizada. Identifica huecos y genera solo las correcciones minimas, una app por respuesta. Incluye prueba manual paso a paso para capturas.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- docs/MATRIZ_CRUD.md
- correcciones puntuales
- tests

DESPUES DE RECIBIR LA RESPUESTA

1. Completar matriz.
2. Aplicar correcciones.
3. Ejecutar toda la suite.

COMANDOS DE VALIDACION

```
python manage.py test
python manage.py check
```

EVIDENCIA

[] Matriz 3x5 completa.

NO AVANZAR SI

Falta una operacion, un template no usa Bootstrap o no hay control de acceso.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-31. No envíe dos prompts de implementacion al mismo tiempo.

7.4 - Recorrido integrado

1. Desde la landing abre login, registro y cada modulo permitido.
2. Desde el panel administrativo entra a los tres CRUD y vuelve sin usar el boton atras como unica navegacion.
3. Cambia manualmente un identificador de URL para comprobar propiedad/permisos.
4. Comprueba que no hay un formulario generico para editar saldo, movimiento o auditoria.

Comandos

```
python manage.py check
python manage.py test
python manage.py runserver
```

Evidencia que debes guardar

- [] Inventario de rutas y matriz CRUD.

**NO PROSIGAS
TODAVIA**

Existe una ruta 404/500, un enlace pages/*.html o un formulario para cambiar saldos directamente.

FASE 8

Desarrollo Plus y reglas de negocio minimas

Implementar una mejora adicional real y demostrar seguridad/comprehension.

Resultado que debe existir al terminar

Login real, grupos/modos, reporte CSV, servicios seleccionados y auditoria de seguridad.

Secuencia intercalada de esta fase

Paso	Accion del estudiante	Prompt / archivo	Comprobacion para seguir
8.1	Implementar autenticacion real.	P-31	Registro/login/logout/password tests.
8.2	Implementar grupos y modo activo.	P-32	Matriz de permisos aprobada.
8.3	Agregar reporte CSV.	P-33	Solo administrador y sin datos sensibles.
8.4	Implementar servicios uno por uno.	P-34	Cada servicio pasa tests antes del siguiente.
8.5	Auditar seguridad.	P-35	Criticos y altos cerrados.

P-31

Autenticacion real Django

Fase / uso

Mejora adicional - Implementar login, registro y logout seguros.

ANTES DE ENVIAR EL PROMPT

[] Custom User/Forms listos.

[] Templates P-18.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Implementa autenticacion real con Django: registro de cliente adulto con terminos, login por username o email, logout POST, cambio de password y redireccion segura. Usa AuthenticationForm/UserCreationForm adaptados, login(), logout(), CSRF y messages. No mostrar usuarios demo ni guardar password en JSON/localStorage. Entrega forms/views/urls/templates/tests solo de accounts auth.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00

VIRTUAL.

- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/accounts/forms.py
- views.py
- urls.py
- templates/accounts/login/register/password
- tests/test_auth.py

DESPUES DE RECIBIR LA RESPUESTA

1. Aplicar cambios.
2. Crear usuario desde registro.
3. Comprobar hash en admin/shell.
4. Probar logout.

COMANDOS DE VALIDACION

```
python manage.py test apps.accounts
python manage.py runserver
```

EVIDENCIA

[] Login/logout/registro.

NO AVANZAR SI

Password visible en DB, logout por GET o registro sin terminos/edad.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-32. No envíe dos prompts de implementación al mismo tiempo.

P-32

Grupos, modos y permisos

Fase / uso

Mejora adicional - Aislar Cliente/Vendedor/Admin.

ANTES DE ENVIAR EL PROMPT

- [] P-31 verde.
- [] Groups seed disponibles.
- [] Dashboards creados.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Implementa CLIENTE, VENDEDOR y ADMINISTRADOR con Django Groups y session["active_mode"]. Cliente puro entra directo; multirrol elige solo roles asignados mediante POST+CSRF. En modo CLIENTE puede usar funciones completas si tiene rol CLIENTE, aunque tambien sea vendedor/admin. En otros modos no compra. Crea policiees/decorators/mixins, change_mode view, context processor/nav y tests de matriz de acceso. Un administrador con boleto no administra ese evento.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/accounts/policiees.py
- decorators.py o mixins.py
- views change_mode
- context_processors.py
- tests/test_permissions.py

DESPUES DE RECIBIR LA RESPUESTA

1. Aplicar codigo.

2. Ejecutar tests por combinacion.
3. Manipular URL manualmente.

COMANDOS DE VALIDACION

```
python manage.py test apps.accounts apps.lottery
```

EVIDENCIA

[] Matriz de modos aprobada.

NO AVANZAR SI

El modo crea roles, usa GET como autoridad o mezcla permisos entre dashboards.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-33. No envíe dos prompts de implementacion al mismo tiempo.

P-33

Reporte CSV filtrado

Fase / uso

Mejora adicional reforzada - Demostrar una característica avanzada útil.

ANTES DE ENVIAR EL PROMPT

[] Login/roles verdes.

[] Datos demo.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Crea un reporte real en Django exportable a CSV para administrador: eventos y boletos filtrados por producto, estado y rango de fechas. Usa StreamingHttpResponse o HttpResponse, cabeceras correctas, QuerySets filtrados y permisos ADMINISTRADOR. No incluir documento, correo, telefono ni saldos sensibles. Agrega formulario de filtros, boton Bootstrap, vista, URL y tests.

RESTRICCIONES DE INTERFAZ QUE NO DEBES PASAR POR ALTO:

- Todas las paginas extienden una base comun y usan {% url %}, {% static %}, messages, CSRF y datos del backend.
- La cabecera muestra logo, nombre Loteria Binaria, aviso de simulacion academica, usuario autenticado y modo activo cuando corresponda.
- La navegacion solo muestra acciones permitidas para el modo activo; no debe haber botones, enlaces ni tarjetas de funciones inexistentes.
- Debe haber estados vacios, mensajes de error claros, confirmaciones para acciones sensibles y diseno responsive a 360, 390, 768, 1024 y 1440 px.
- Usar componentes Bootstrap reales: navbar/offcanvas, grid, cards, forms, alerts, badges, table-responsive, pagination y modal cuando corresponda.
- Conservar la identidad azul profundo/dorado del ZIP sin copiar reglas obsoletas de 5%, 15%, 75%, hexadecimal de 4 o localStorage de negocio.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/lottery/reports.py o views.py
- forms de filtros
- template report
- tests/test_reports.py

DESPUES DE RECIBIR LA RESPUESTA

1. Aplicar.
2. Descargar CSV.
3. Abrir y revisar columnas.

COMANDOS DE VALIDACION

```
python manage.py test apps.lottery
```

EVIDENCIA

[] CSV descargado y captura.

NO AVANZAR SI

El reporte expone datos personales o es accesible por cliente/vendedor.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-34. No envíe dos prompts de implementación al mismo tiempo.

P-34

Servicios de negocio minimos

Fase / uso

Ampliacion coherente - Mover reglas fuera de views.

ANTES DE ENVIAR EL PROMPT

[] CRUD y auth verdes.

[] Elegir solo flujos necesarios para demo.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Diseña e implementa un servicio de dominio por respuesta. Orden: recarga REAL 1:1; conversion VIRTUAL->REAL 10%; compra mayorista 0.90/1.00; crear solicitud con reserva; compra de boleto. Usa transaction.atomic, montos enteros, operation_id, validaciones del servidor y Movement. Para SQLite aclara que select_for_update no prueba concurrencia; esa prueba se realiza en MySQL. Entrega services.py y tests, no interfaces adicionales hasta que el servicio pase.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/finance/services.py
- apps/vendors/services.py
- apps/lottery/services.py
- tests de servicios

DESPUES DE RECIBIR LA RESPUESTA

1. Implementar un servicio.
2. Ejecutar tests.
3. Confirmar rollback.
4. Solo despues pedir el siguiente.

COMANDOS DE VALIDACION

```
python manage.py test apps.finance apps.vendors apps.lottery
```

EVIDENCIA

[] Servicio y rollback verificado.

NO AVANZAR SI

Se calculan montos en float, la view modifica saldos o se implementan todos los flujos en una respuesta.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-35. No envíe dos prompts de implementación al mismo tiempo.

P-35

Revisión de seguridad

Fase / uso

Antes de MySQL - Cerrar fallas básicas.

ANTES DE ENVIAR EL PROMPT

[] Toda funcionalidad SQLite lista.

[] Adjuntar settings, views y forms.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Audita seguridad del taller: CSRF, POST para cambios, login/role/mode/ownership, password hash, .env, mensajes sin trazas, no delete historico, no datos de tarjeta, datos publicos minimizados y ALLOWED_HOSTS/DEBUG local. Busca manipulacion de pk, role, saldo, precio y estado. Entrega hallazgos por severidad y parches minimos con tests. No agregues dependencias innecesarias.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- docs/AUDITORIA_SEGURIDAD.md
- parches
- tests

DESPUES DE RECIBIR LA RESPUESTA

1. Aplicar criticos/altos.
2. Ejecutar tests.
3. Buscar secretos en Git.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py test
git grep -n "SECRET_KEY\|DATABASE_URL\|password="
```

EVIDENCIA

[] Auditoria cerrada.

NO AVANZAR SI

Hay secretos versionados, POST sin CSRF o rutas sin propiedad/permisos.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-36. No envíe dos prompts de implementación al mismo tiempo.

8.6 - Orden obligatorio para P-34

1. Solicita y completa un solo servicio por respuesta.
2. Primero recarga REAL 1:1 y ejecuta sus tests.
3. Luego conversion VIRTUAL->REAL 10% y ejecuta sus tests.
4. Despues compra mayorista 0.90/1.00.
5. Luego solicitud con reserva REAL y finalmente compra de boleto.
6. No agregues una interfaz para un servicio hasta que el backend y rollback esten verdes.

Evidencia que debes guardar

[] Capturas de login/modo/reporte y salida de tests de servicios.

**NO PROSIGAS
TODAVIA**

La IA entrega todos los servicios de una vez, las views calculan saldos o la navegacion muestra funciones de otro modo.

FASE 9

Migracion de SQLite a MySQL

Cumplir la segunda fase de base de datos sin perder datos ni reescribir migraciones.

Resultado que debe existir al terminar

MySQL configurado, esquema creado por Django, datos importados, conteos comparados y CRUD probado.

Secuencia intercalada de esta fase

Paso	Accion del estudiante	Prompt / archivo	Comprobacion para seguir
9.1	Preparar driver, base y usuario MySQL.	P-36	Conexion verificada sin tablas manuales finales.
9.2	Exportar datos desde SQLite.	P-37	Fixture limpia y conteos.
9.3	Cambiar settings y ejecutar migrate.	P-38	SELECT DATABASE()/VERSION() confirma MySQL.
9.4	Importar y comparar datos.	P-39	Conteos iguales y CRUD.
9.5	Resolver incompatibilidades solo si aparecen.	P-40	Migracion nueva; nunca --fake.

P-36

Preparar MySQL

Fase / uso

Parte 2 - Instalar driver y crear base/usuario final.

ANTES DE ENVIAR EL PROMPT

[] SQLite completamente verde.

[] Backup db.sqlite3.

[] MySQL 8 iniciado.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Dame pasos exactos para Windows y MySQL Workbench: verificar MySQL, instalar mysqlclient compatible con Django 5.2, crear base loteria_taller3 utf8mb4, crear usuario local con privilegios solo sobre esa base, y probar conexion. No crear tablas manualmente en la base final. Incluye consultas de verificacion y como ocultar la clave en capturas.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- requirements.txt
- base/usuario MySQL
- .env.example actualizado

DESPUES DE RECIBIR LA RESPUESTA

1. Instalar driver dentro de .venv.
2. Crear base/usuario.
3. Probar conexion.
4. Actualizar requirements.

COMANDOS DE VALIDACION

```
pip install "mysqlclient>=2.2.1"
pip freeze > requirements.txt
mysql --version
```

EVIDENCIA

[] Workbench y driver instalado.

NO AVANZAR SI

Se crean tablas manuales en loteria_taller3 o la clave se escribe en Git/capturas.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-37. No envíe dos prompts de implementación al mismo tiempo.

P-37

Exportar datos SQLite

Fase / uso

Migracion DB - Crear fixture limpia antes del cambio.

ANTES DE ENVIAR EL PROMPT

- [] Servidor detenido.
- [] SQLite activa.
- [] Conteos registrados.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Revisa los modelos y genera el comando dumpdata correcto para exportar datos de negocio desde SQLite, excluyendo sessions/admin logs y evitando dependencias rotas. Debe incluir auth.group si los usuarios tienen grupos, usar natural foreign/primary y --output para evitar problemas de codificacion PowerShell. Crea tambien un comando o shell de conteos antes/despues. No cambies a MySQL aun.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufixo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- data/sqlite_export.json
- evidencias/05_mysql/conteos_sqlite.txt

DESPUES DE RECIBIR LA RESPUESTA

1. Ejecutar dumpdata.
2. Abrir JSON y verificar que no hay passwords en texto ni secretos.
3. Guardar hash/tamano.

COMANDOS DE VALIDACION

```
python manage.py dumpdata accounts core finance vendors lottery auth.group --natural-foreign --
natural-primary --indent 2 --output data/sqlite_export.json
python manage.py check
```

EVIDENCIA

- [] Fixture y conteos.

NO AVANZAR SI

El archivo esta vacio, incluye sesiones innecesarias o se exportan credenciales en texto.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continue con P-38. No envie dos prompts de implementacion al mismo tiempo.

P-38

Cambiar settings y migrar MySQL

Fase / uso

Migracion DB - Crear esquema final mediante ORM.

ANTES DE ENVIAR EL PROMPT

[] P-36/P-37 verdes.

[] Copia de .env SQLite guardada fuera de Git.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Revisa config/settings.py y genera el cambio minimo de .env para DATABASE_URL MySQL. Configura charset utf8mb4 y STRICT_TRANS_TABLES. Indica comandos para check, migrate --plan, migrate, showmigrations y shell SELECT DATABASE()/VERSION(). No uses migrate --fake ni edites migraciones antiguas. Si falla, solicita el error exacto.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- .env local MySQL
- tablas creadas por migrations

DESPUES DE RECIBIR LA RESPUESTA

1. Cambiar DATABASE_URL.
2. Ejecutar plan y migrate.
3. Verificar database/version.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py migrate --plan
python manage.py migrate
python manage.py showmigrations
python manage.py shell -c "from django.db import connection; c=connection.cursor();
c.execute('SELECT DATABASE(), VERSION()'); print(c.fetchone())"
```

EVIDENCIA

[] Migrations MySQL verdes.

NO AVANZAR SI

El sitio sigue conectado a SQLite, hay --fake o se modifican migraciones aplicadas.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-39. No envíe dos prompts de implementación al mismo tiempo.

P-39

Importar y comparar datos

Fase / uso

Migracion DB - Completar la migracion sin perdida.

ANTES DE ENVIAR EL PROMPT

[] MySQL migrada vacia.

[] Fixture P-37.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Dame el orden seguro para cargar data/sqlite_export.json en MySQL, resolver dependencias de grupos/permisos sin duplicar catalogos, comparar conteos y probar siguiente ID. Incluye loaddata, chequeos y rollback/limpieza solo de la base MySQL si la carga falla antes de entregar. No toques db.sqlite3.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- datos cargados MySQL
- conteos_mysql.txt

DESPUES DE RECIBIR LA RESPUESTA

1. Ejecutar loaddata.
2. Comparar conteos por modelo.
3. Crear y borrar un registro de prueba seguro.
4. Probar login/CRUD.

COMANDOS DE VALIDACION

```
python manage.py loaddata data/sqlite_export.json
python manage.py check
python manage.py test
```

EVIDENCIA

[] Conteos iguales y CRUD.

NO AVANZAR SI

Faltan registros, hay IDs duplicados o se elimina la copia SQLite.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-40. No envíe dos prompts de implementación al mismo tiempo.

P-40

Reparar incompatibilidades SQLite/MySQL

Fase / uso

Soporte DB - Corregir portabilidad sin romper historia.

ANTES DE ENVIAR EL PROMPT

[] Pegar error completo y archivo relevante.

[] No borrar migraciones.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Diagnostica este error de migracion/ejecucion MySQL usando los archivos y traceback completos. Revisa collation, longitud de indices, constraints, valores datetime, JSONField, defaults, SQL mode y on_delete. Propone la correccion minima mediante una migracion nueva y codigo portable SQLite/MySQL. No uses --fake, no edites migraciones aplicadas y no recomiendes recrear la base sin explicar impacto y obtener autorizacion.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- parche puntual
- migracion nueva si aplica
- test de regresion

DESPUES DE RECIBIR LA RESPUESTA

1. Aplicar el parche.
2. Ejecutar migrate en SQLite de copia y MySQL.
3. Ejecutar tests.

COMANDOS DE VALIDACION

```
python manage.py makemigrations
python manage.py migrate
python manage.py test
```

EVIDENCIA

[] Error reproducido y corregido.

NO AVANZAR SI

La solucion oculta el error con --fake o modifica una migracion ya aplicada.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continue con P-41. No envie dos prompts de implementacion al mismo tiempo.

9.6 - Evidencia obligatoria de MySQL

1. Captura de MySQL Workbench con la base final y sus tablas creadas por migrate.
2. Captura de `SELECT DATABASE(), VERSION()` desde Django shell.
3. Comparacion de conteos SQLite y MySQL por modelo.
4. Login, admin y un ciclo CRUD completo ejecutado sobre MySQL.
5. Mantener db.sqlite3 como respaldo de la primera fase; no subir secretos de MySQL.

Evidencia que debes guardar

[] Workbench, terminal y CRUD MySQL.

**NO PROSIGAS
TODAVIA**

La base final fue creada con DDL manual, se uso --fake, faltan datos o el proyecto sigue conectado a SQLite.

FASE 10

Pruebas, evidencias y entrega

Cerrar la rubrica con pruebas repetibles y un paquete limpio.

Resultado que debe existir al terminar

Suite verde, smoke test, Word con capturas, README, evidencia de IA, auditoria final y comprimido verificado.

Secuencia intercalada de esta fase

Paso	Accion del estudiante	Prompt / archivo	Comprobacion para seguir
10.1	Completar suite automatizada.	P-41	Tests por app y suite completa.
10.2	Ejecutar smoke test manual.	P-42	20-30 pasos aprobados.
10.3	Preparar Word de evidencias.	P-43	Capturas reales con descripcion.
10.4	Completar README y uso de IA.	P-44	Proyecto reproducible.
10.5	Auditar y comprimir.	P-45	Paquete abre en carpeta limpia.

P-41

Suite automatizada final

Fase / uso

Parte pruebas - Cubrir rubrica y reglas criticas.

ANTES DE ENVIAR EL PROMPT

[] MySQL funcionando.

[] Tres CRUD y mejora lists.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Audita y completa la suite de pruebas con TestCase para modelos/forms/views/permisos y TransactionTestCase solo donde MySQL permita concurrencia. Cobertura minima: tres CRUD; User/edad/unicidad; roles/modos; eliminacion protegida; productos 4/5/6 unicos; cierre 10 min; recarga 1:1; conversion 10%; compra vendedor 0.90; propiedad de datos; CSV sin datos sensibles. Genera tests por app, no un archivo gigante.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- apps/*/tests/
- docs/RESULTADOS_PRUEBAS.md

DESPUES DE RECIBIR LA RESPUESTA

1. Ejecutar app por app.
2. Ejecutar suite completa.

3. Guardar salida.

COMANDOS DE VALIDACION

```
python manage.py test apps.accounts
python manage.py test apps.vendors
python manage.py test apps.lottery
python manage.py test
```

EVIDENCIA

☐ Suite final verde.

NO AVANZAR SI

Hay tests omitidos, dependientes del orden o solo pruebas manuales.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-42. No envíe dos prompts de implementación al mismo tiempo.

P-42

Smoke test manual completo

Fase / uso

Parte pruebas - Recorrer el sistema como evaluador.

ANTES DE ENVIAR EL PROMPT

- [] Suite verde.
- [] Datos demo MySQL.
- [] Navegadores listos.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Crea un guion de prueba manual de 20-30 pasos para evaluacion: entorno, login, admin, CRUD accounts/vendors/lottery, validaciones, delete protegido, dashboards por rol, reporte CSV, responsive, logout y confirmacion MySQL. Incluye dato de entrada, resultado esperado, captura requerida y casilla aprobado/falla. No agregues funciones no implementadas.

RESTRICCIONES DE MODELOS Y NEGOCIO:

- User personalizado debe existir antes de la primera migracion general.
- Montos en BigIntegerField con sufijo _minor; nunca float.
- Relaciones historicas usan PROTECT o desactivacion logica; no borrar movimientos, boletos, resultados, aceptaciones ni auditoria.
- Las reglas criticas se ejecutan en services.py con transaction.atomic; las vistas no duplican formulas ni transiciones.
- El codigo debe ser portable entre SQLite y MySQL; evita funciones exclusivas de PostgreSQL.
- OCTAL: 0-7, 4 simbolos unicos. DECIMAL: 0-9, 5 unicos. HEXADECIMAL: 0-9/A-F, 6 unicos.
- Recarga REAL simulada 1:1; conversion VIRTUAL a REAL con 10%; compra mayorista 0.90 REAL por 1.00 VIRTUAL.
- En modo CLIENTE se puede comprar si el rol CLIENTE fue asignado, aunque la cuenta tambien tenga otros roles. En modo VENDEDOR o ADMINISTRADOR no se compra.

ARCHIVOS ESPERADOS

- docs/SMOKE_TEST.md
- evidencias ordenadas

DESPUES DE RECIBIR LA RESPUESTA

1. Ejecutar todos los pasos.
2. Anotar fallas y corregir antes de capturar final.
3. No manipular datos en admin para ocultar errores.

COMANDOS DE VALIDACION

```
python manage.py runserver
```

EVIDENCIA

- [] Smoke test firmado/fechado.

NO AVANZAR SI

Alguna ruta principal da 404/500 o una captura no coincide con el resultado esperado.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-43. No envíe dos prompts de implementación al mismo tiempo.

P-43

Estructura del Word de evidencias

Fase / uso

Entrega - Documentar lo realizado con capturas.

ANTES DE ENVIAR EL PROMPT

[] Capturas completas organizadas.

[] Enunciado/rubrica visibles.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Genera la estructura de un documento Word de evidencias para el Taller #3. Debe incluir portada, apreciacion personal de IA, ER/apps, Parte 1 entorno/proyecto/admin, Parte 2 SQLite->MySQL, Parte 3 tres CRUD, Parte 4 Bootstrap, mejora login/roles/reporte, pruebas y conclusion. Para cada captura crea titulo y una breve descripcion de lo que debe demostrar. No inventes resultados ni capturas.

ARCHIVOS ESPERADOS

- Documento Word de evidencias
- indice de capturas

DESPUES DE RECIBIR LA RESPUESTA

1. Crear el Word.
2. Insertar capturas reales.
3. Escribir descripciones propias.
4. Verificar legibilidad.

EVIDENCIA

[] Word completo.

NO AVANZAR SI

Faltan MySQL, tres CRUD, Bootstrap o apreciacion personal.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con P-44. No envíe dos prompts de implementacion al mismo tiempo.

P-44

README y evidencia del uso de IA

Fase / uso

Entrega - Demostrar comprension y reproducibilidad.

ANTES DE ENVIAR EL PROMPT

[] Sistema final MySQL.

[] Prompts guardados.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir codigo, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, codigo completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Revisa el proyecto final y redacta un README para que otra persona lo ejecute: requisitos, crear/activar .venv, instalar requirements, configurar .env.example, SQLite de primera fase, crear MySQL, migrate, seed, superuser, runserver, usuarios demo no sensibles, rutas, tests, estructura y limitaciones academicas. Agrega una seccion "Uso de IA" con prompts por etapa, decisiones revisadas y ajustes personales, sin afirmar que la IA hizo todo.

ARCHIVOS ESPERADOS

- README.md
- .env.example
- docs/USO_IA.md

DESPUES DE RECIBIR LA RESPUESTA

1. Probar README en una carpeta limpia.
2. Eliminar passwords fijos.
3. Revisar comandos.

COMANDOS DE VALIDACION

```
pip install -r requirements.txt
python manage.py check
python manage.py test
```

EVIDENCIA

[] README reproducible.

NO AVANZAR SI

El README incluye secretos, comandos inexistentes o oculta el uso de IA.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continue con P-45. No envíe dos prompts de implementacion al mismo tiempo.

P-45

Auditoria final, nota y paquete

Fase / uso

Cierre - Comparar contra 100 puntos y preparar entrega.

ANTES DE ENVIAR EL PROMPT

[] Word, código, MySQL y tests listos.

[] Nombre de archivo verificado.

PROMPT PARA COPIAR

RECORDATORIO OBLIGATORIO PARA ESTA RESPUESTA:

- Este es el Taller #3 de Loteria Binaria con Django, no el proyecto empresarial ni el monorepo NestJS.
- Respeta este orden de autoridad: enunciado del taller; cinco documentos canonicos; guia docente VideoClub; ZIP antiguo solo como referencia visual.
- La entrega usa SQLite en la primera fase y MySQL en la segunda. No uses PostgreSQL en este taller.
- La interfaz debe usar Bootstrap 5.3 de forma real; el CSS propio solo complementa la identidad azul oscuro y dorada.
- No generes todo el proyecto. Trabaja solo la tarea solicitada y conserva todo lo que ya funciona.
- Antes de escribir código, revisa los archivos actuales que te entregue. No inventes rutas, modelos, campos, imports ni funciones previas.
- Devuelve: diagnostico breve, rutas exactas, código completo solo de los archivos solicitados, comandos, pruebas y criterio para continuar.
- No edites manage.py salvo que exista una razon extraordinaria y explicada. Normalmente se usa desde terminal.
- No uses JSON, localStorage, valores del navegador ni JavaScript como fuente de verdad de usuarios, roles, saldos, boletos, solicitudes o resultados.
- No agregues pagos reales, tarjetas, API REST, Celery, Redis, microservicios ni funciones fuera del alcance sin autorizacion.

Audita la entrega final contra la rubrica de 100 puntos. Revisa fuentes, requirements, migrations, tres CRUD, Bootstrap, MySQL, login/roles/reporte, tests, README, Word y capturas. Entrega tabla puntaje obtenido/evidencia/falta, problemas bloqueantes y correcciones minimas. Despues genera checklist para comprimir sin .venv, .env, __pycache__, secretos ni archivos temporales. No declares 10/10 si falta evidencia.

ARCHIVOS ESPERADOS

- docs/AUDITORIA_FINAL.md
- HERRERA_NIETO_T3.rar o nombre confirmado

DESPUES DE RECIBIR LA RESPUESTA

1. Corregir bloqueantes.
2. Ejecutar test final.
3. Comprimir.
4. Extraer en otra carpeta y verificar apertura.
5. Subir antes de las 23:00.

COMANDOS DE VALIDACION

```
python manage.py check
python manage.py test
git status
```

EVIDENCIA

[] Auditoria final y archivo comprimido.

NO AVANZAR SI

Falta cualquier criterio obligatorio, el paquete no abre o contiene .env/.venv.

SIGUIENTE PASO

Cuando los comandos esten correctos, la evidencia este guardada y el bloque de NO AVANZAR SI no se cumpla, continúe con la auditoria final, el documento de evidencias y el paquete de entrega. No envíe dos prompts de implementacion al mismo tiempo.

Checklist final de salida

- [] Entorno virtual creado, activado y documentado; desactivacion capturada al finalizar.
- [] Proyecto y cinco apps creadas; tres apps con CRUD completo.
- [] Custom User definido antes de la primera migracion.
- [] SQLite funcional y documentada.
- [] MySQL funcional, con datos migrados y CRUD probado.
- [] Django Admin funcional.
- [] Bootstrap 5.3 visible en navbar, grid, cards, forms, tables, alerts, badges y responsive.
- [] Login, grupos/modo activo y reporte CSV como mejora adicional.
- [] Reglas antiguas contradictorias eliminadas de textos y codigo.
- [] Suite automatizada y smoke test verdes.
- [] Word con capturas completas a pantalla compartida y breve descripcion.
- [] README y documento de uso de IA con apreciacion personal.
- [] Comprimido sin .venv, .env, __pycache__, secretos ni copias temporales.

Fin del manual

No se considera terminado solo porque runserver abre. La entrega termina cuando SQLite y MySQL estan evidenciadas, los tres CRUD y la mejora funcionan, las pruebas pasan y el paquete se abre correctamente en una carpeta limpia.